

(1) (a) Error Identification:

Line No.	Error	Description.
5	for(i=0 ; i<=n ; i++)	Loop executes one extra iteration, causing array out-of-bound access.
11	int arr[5] = {1,2,3,4,5}	Missing Semicolon (;)
13	total = sumArray(arr)	Function called with one argument instead of two (arr and n)
15	printf("Correct sum\n")	Missing Semicolon after printf
18	int c = a/b;	Division by zero.
20	*p=20;	uninitialized pointer dereference.

(b) Classification of Errors:

1. Lexical Errors:

No lexical errors are present.

2. Syntax Errors:

* Missing Semicolon after array declaration.

* Missing Semicolon after printf() statement.

3. Semantic Errors:

* Incorrect function call (sumArray(arr) instead of sumArray(arr,5)).

4. Logical Errors:

Loop condition $i < n$ should be $i <= n$ otherwise, one extra element is accessed.

5. Runtime Error:

Division by zero (a/b).

Dereferencing an uninitialized pointer (*p=20)

(C) Compilation Behaviour:

Compile Time Errors:

- * Missing semicolon after array declaration
- * Missing semicolon after printf()
- * Function argument mismatch.

Execution Time Errors:

- * Division by zero
- * Invalid pointer Access
- * Array index out of bound due to incorrect loop.

How Early Errors affect compilation?

Lexical and syntax errors prevent the compiler from parsing the remaining program correctly. Therefore, runtime errors such as division by zero or invalid pointer access cannot be detected until syntax errors are fixed.

(d) Corrected Program:

```
#include <stdio.h>
int sumArray (int arr[], int n)
{
    int i, sum = 0;
    for (i = 0; i < n; i++)
    {
        sum += arr[i];
    }
    return sum;
}

int main()
{
    int arr[5] = {1, 2, 3, 4, 5};
    int total;
```

```
total = sumArray (arr, 5);
if (total == 15)
{
    printf("Corrected Sum \n");
}
int a = 10, b = 2;
int c = a / b;
printf("Division = %d \n", c);
int value = 20;
int *p = &value;
printf("Pointer Value = %d \n", *p);

return 0;
}
```

(c) Impact Analysis:

1. Improper pointer Usage:

Accessing an uninitialized pointer leads to segmentation fault or program crash.

2. Division by zero:

- * Causes runtime Exception.

- * Program terminates Abnormally.

3. Incorrect Loop Condition:

- * Access memory outside the array.

- * Produces incorrect output or undefined behaviour.

(d) Advanced Compiler Perspective:

Compiler Improvements:

1. Better Error Messages:

- * Display exact line no.

- * Suggest possible corrections.

- * Highlight the incorrect token.

ex:

Error: Missing ';' after declaration.

Suggestion: Add ';' at the end of line 11.

2. Early detection of Runtime Errors:

- * Warn about possible division by zero.

- * Warn when an uninitialized pointer is dereferenced.

- * Detect potential array index out-of-bounds using static analysis.

Enhancements in Compiler Design:

1. Static Analysis:

Detect null pointers, divide-by-zero, and buffer overflows before execution.

2. Intelligent error Recovery:

Continue compiling after minor syntax errors so that multiple errors are reported in one compilation, reducing debugging time.

(2) (a) Error Identification:

Line	Invalid Token	Error:	
3	1num	Identifier starts with a digit	
4	rat@	Invalid character @ in Identifier	
'5'	'abc'	Character literal contains more than one character	
6	50 \$20	Invalid symbol \$	
7	num#1	Invalid symbol # in Identifier.	
8	% count	Identifier begins with %.	
9	"Welcome to Compiler Design"	Missing closing double quotation mark.	
10	09	Invalid octal constant (digit 9 not allowed)	
11	0x212	16	
12	12.3.4	17	
13	"Hello	18	10 @ 5
#16	Invalid id	19	'xy'
15 17	5..6	20	3 value

(b) Error Classification:

Invalid Token	Type of Rule Violated.
1 num , 3 value	Identifier Rule.
rate@ , num#1 , Invalid-id , %count	Identifier / operator rule
09 , 0x212 , 12.3.4 , 5..6	Constant formation rule
"Welcome to Compiler Design , "Hello	String Literal rule
'abc' , 'xy'	Character Literal rule.
50\$20 , 10@5	Illegal Symbol / Operator rule.

Why these are Lexical errors?

A lexical error occurs when the scanner cannot form a valid token because it violates the language's token formation rules.

(c) Token Correction:

Invalid Token	Correct Token	Invalid	Correct	Invalid	Correct
1 num	num 1	"Welcome to Compiler Design	"Welcome to Compiler Design"	5..6	5.6
rate@	rate	09	9	10@5	10 + 5
'abc'	'a'	0x212	0x12	'xy'	'x'
50 \$ 20	50 + 20	12.3.4	12.34	3value	value 3
num#1	num 1	"Hello	"Hello"		
%count	count	Invalid-id	Invalid-id.		

(c) Compiler Behaviour Analysis:

ii) How does the lexical analyzer process errors?

The lexical analyzer:

1. Reads the source program character by character.
2. Groups character into valid tokens.
3. Compares tokens with lexical rules.
4. Reports an error whenever an invalid token is found.
5. Continues scanning using error recovery whenever possible.

iii) Does the compiler stop immediately?

No.

Modern compilers generally use error recovery techniques so that they can continue scanning after detecting a lexical error. This allows multiple errors to be reported in a single compilation, making debugging easier.

(e) Advanced Thinking

ii) DFA Rule to Detect Invalid Identifiers:

A valid identifier must:

- * Begin with a letter (A-Z, a-z) or underscore (_)
- * Be followed by letters, digits or underscores.

Simple DFA idea:

* Start State (S0):

- letter or _ → Valid Identifier state (S1)
- Digit → Error state.
- Special Symbol → Error state.

* State S1

→ letter / digit / _ → stay in S2

→ Any other symbol → Identifier completed.

iii) Suggestion to Improve Error Message

1. Display The exact line no where the error occurs
2. Highlight The invalid token in the source code.
3. Explain the reason for the error in simple language.
4. Suggest a possible correction.

ex:

Error at Line 2;

Invalid Identifier 'inum'.

Reason:

Identifiers cannot begin with a digit.

Suggestion;

Rename it as 'num1'.

(3) (a) Identify Errors:

Line No.	Error	Description	Line No.	Error	Description
3	return a/b	Missing semicolon(;)	11	printf("Result is %d\n")	Missing semicolon
6	int #value = 20	Invalid Identifier containing #	15	printf("%d\n", arr[10])	Array index out of bounds (valid indices are 0-9)
7	float num = 15.5	Missing semicolon	17	*ptr = 25;	Referencing an uninitialized pointer
9	result = del(10);	Func. call with one argument instead of two.	19	total = x + y + result + 0	Redundant + 0
10	if (result > 5)	Assignment operator (=) used instead of comparison	20-22	while (x < 20)	Infinite loop because x is never incremented

(b) Classification by Compiler:

(i) Lexical Errors:

- * `num` → Invalid Identifiers

(ii) Syntax Errors

- * Missing semicolon after `return a/b`.
- * Missing semicolon after `float num = 15.5`
- * Missing semicolon after `printf()`.

(iii) Semantic Errors

- * Function call `divide(10)` has incorrect no. of arguments.
- * `Pf(result = 5)` uses `=` assignment instead of comparison.

(iv) Intermediate Code Issues:

- * Incorrect comparison due to assignment operator changes program logic.

- * Expression evaluation may generate incorrect intermediate code.

(v) Optimization Issues:

- * `+0` is a redundant expression and can be removed.

- * variable `num` is declared but never used.

(vi) Runtime Errors:

- * Division by zero
- * Array index out of bounds.
- * Dereferencing an uninitialized pointer.
- * Infinite loop.

(c) Compilation vs Execution:

Compile - Time Errors:

- * Invalid Identifier (if value)
- * Missing semicolons
- * Incorrect function arguments.

Execution - Time Errors:

- * Division by zero
- * Invalid pointer access
- * Array out of bounds.
- * Infinite loops.

Why some Errors prevent compilation?

Lexical & syntax errors prevent the compiler from generating the syntax tree. Therefore, later phases such as semantic analysis, optimization & code generation cannot proceed until these errors are corrected.

(d) Corrected Program:

```
#include <stdio.h>
int divide (int a, int b)
{
    if (b == 0)
    {
        printf ("Division by zero is
not allowed. \n");
        return 0;
    }
    return a / b;
}

int main()
{
    int value = 20;
```

```
float num = 15.5;
int result;
result = divide(10, 2);
if (result == 5)
{
    printf ("Result is 5\n");
}
int x = 10
int y = 2
int z = x / y;
int arr[4] = {1, 2, 3, 4};
printf ("x = %d\n", arr[3]);
```

```

int main = 0;
int *ptr = &main;
printf("main", *ptr);
int total;
total = *ptr * result;
while (a < 100)
{
    printf("main", a);
    a++;
}
printf("total = %d\n", total);
return 0;
}

```

(c) Deep Analysis:

(i) Effect of Incorrect pointer Usage:

- * An uninitialized pointer points to an unknown memory location.

- * Dereferencing it may cause a segmentation fault, memory corruption, or program crash.

(ii) Impact of Array Bounds Violation

- * Accessing arr[4] exceeds the valid range of the array.

- * This results in undefined behavior, which may produce incorrect values or crash the program.

(iii) Effect of Operator Precedence Errors:

- * Using = instead of == changes the meaning of the condition.

- * The assignment sets result to 5, making the condition always true.

(f) Advanced Task!

I. Improvement In Compiler Design

A. Detect Runtime Errors Earlier:

- * use static analysis to identify possible division by zero and null/uninitialized pointer access before execution.

- * Implement array bounds checking to warn about potential out-of-range accesses.

B. Improve Debugging Messages:

- * Display the exact line no. & highlight the erroneous token.

- * Provide meaningful suggestions to help the programmer fix the error.

ex: Error at line 9

Function 'divide()' expects 2 arguments but only 1 was provided.

Suggestion!

use divide(10, 2);

II Optimization phase:

The compiler can eliminate redundant expressions using constant folding and dead code elimination.

ex: $total = x + y * result + 0;$

Optimized to: $total = x + y * result;$

It can also remove unused variables such as num if they are not referenced.